

Header

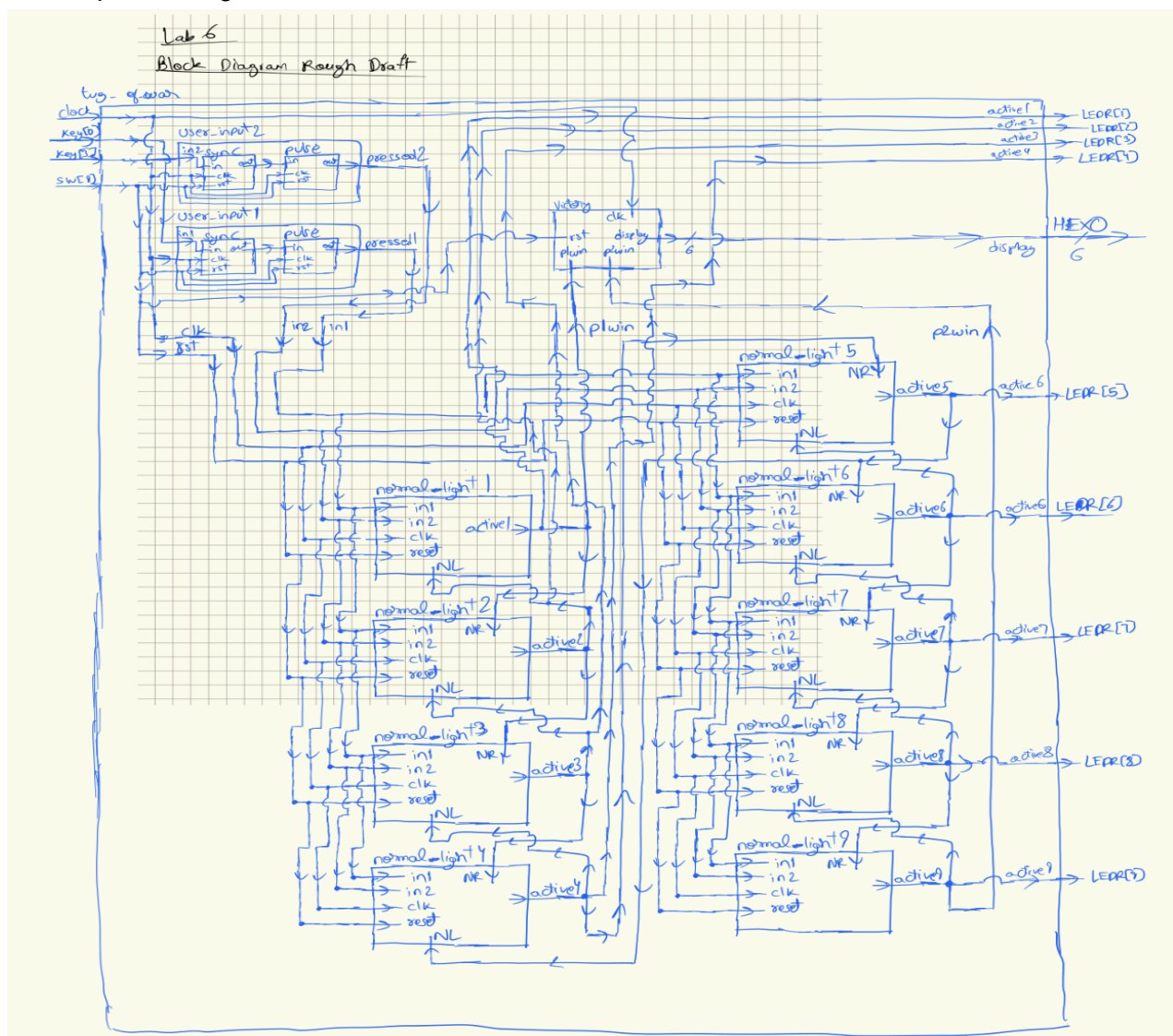


Lab Number: 6

Lab Name: Tug of War

Q1. Top level diagram

Initial top level diagram



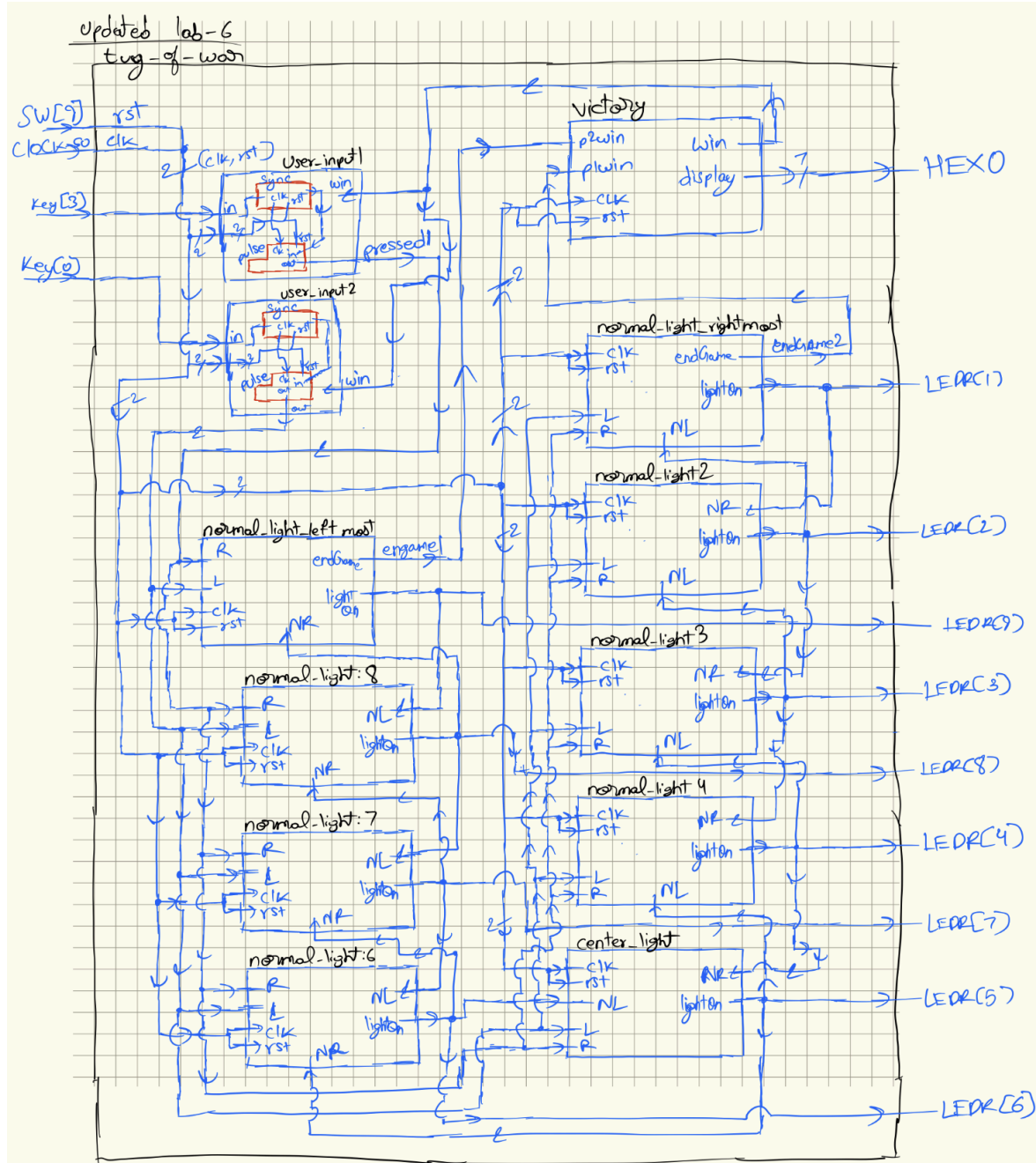
The above screenshot show my initial top level diagram that mostly encapsulates all the functionality as required in the spec. Key inputs are processed by user_input modules which

each have a sync module which accounts for metastability and a pulse module to generate the actual key press inputs as singular pulses.

The processed user inputs alongside clk and reset signals are fed into the fsm's for each light. In this version, I thought I would be able to easily create a universal light module that could account for the edge (literal) cases as well as the center one - however, this did not work out. After the light FSMs process their respective inputs, top-level module reflects the outputs of those FSMs by turning on or keeping off the respective LEDRSs.

For LEDRs 1 and 9, their state is actually fed into the victory module which displays the respective win state when either of them is activated by displaying the number of the player who won on a hex display.

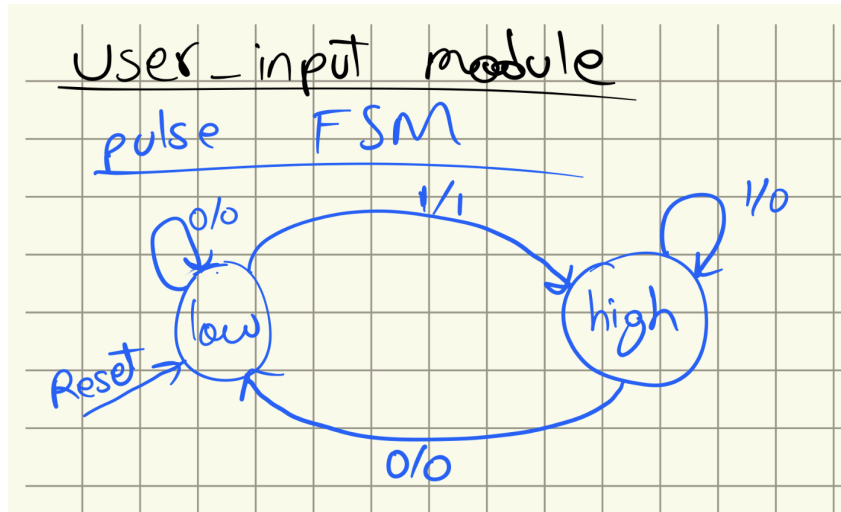
The next page has the updated top level module diagram. Modelsim towards the end of the document.



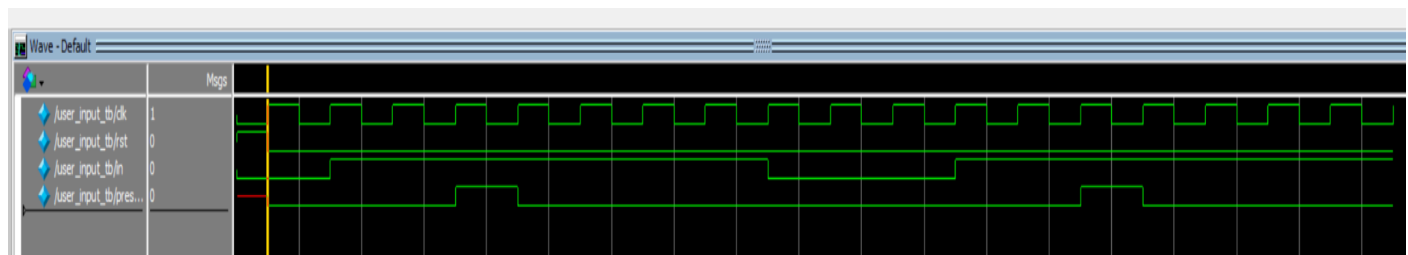
The overall module is very similar, but slight modifications were made as I implemented the diagram. First change was that I created separate leftmost, rightmost, and center normal_light modules - this simply allowed me to be able to change the behaviors in a more modular and predictable way - but honestly did increase the amount of work, so perhaps not the best idea. Then, in this diagram, I opted to combine the rst and clk signals to condense the diagram more - the functionality being communicated is the same, it just makes the diagram look nicer. The biggest change was victory activation and handling. Initially, victory would be detected if the last light turned on, but now (more aligned with the spec) victory only happens when the "rope" "falls off" of the playfield - when the edge lights turn off right after another input in their

respective directions. Furthermore, victory module outputs a signal win that indicates to the user module to stop taking in inputs.

Q2. User input module

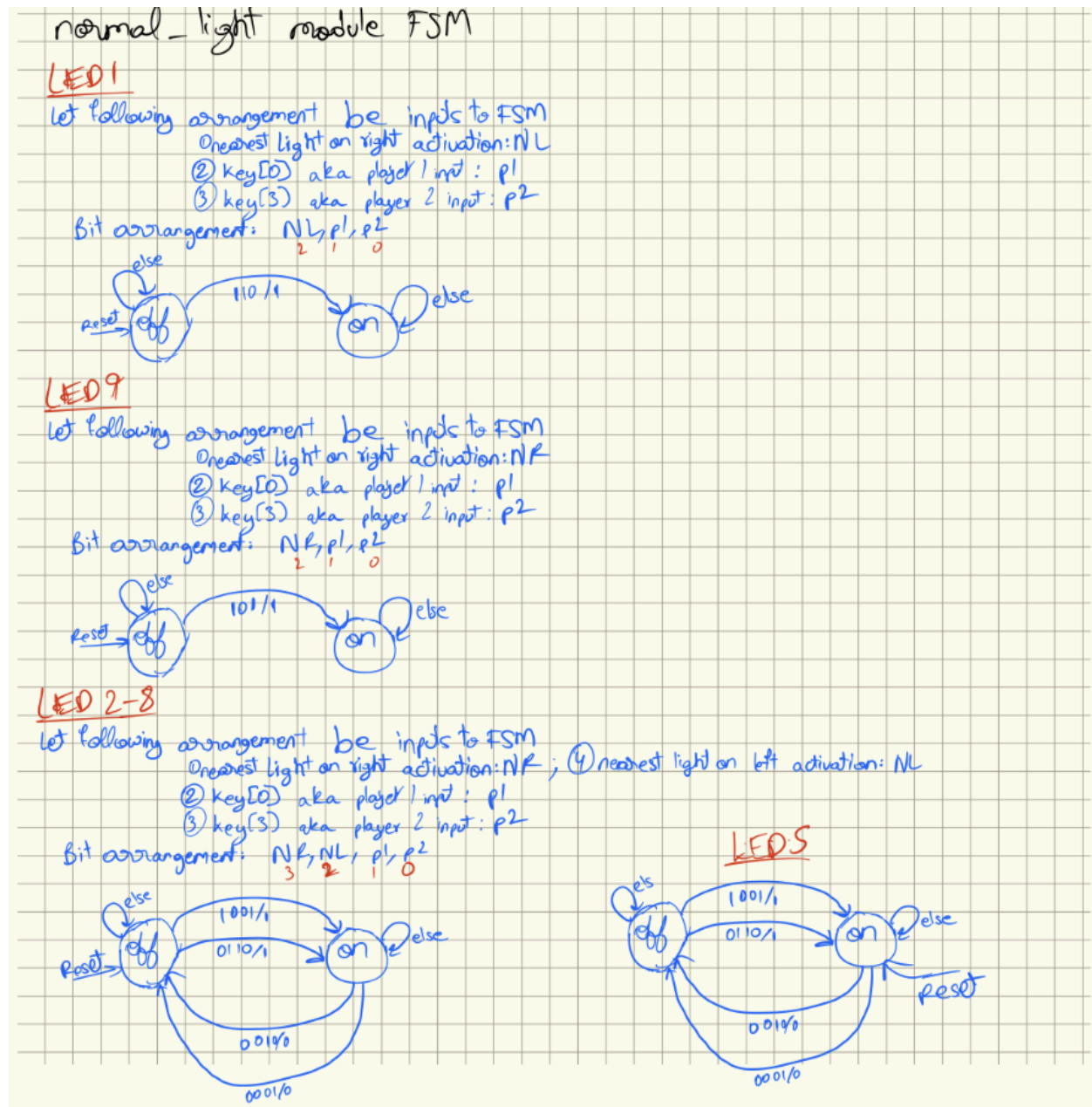


This is the FSM for the pulse generator. A pulse is only generated for a singular input. Thus, when we start at low and get a signal of 1, we output 1 and transition to the high state. At high state, we do not output anything even if we receive one because this is equivalent to holding the key down, which is not what we want. So, we transition back to low state one from high state when no more input is being inputted by player - captures pulse behavior of a single key press corresponding to 1 signal, not continuous.



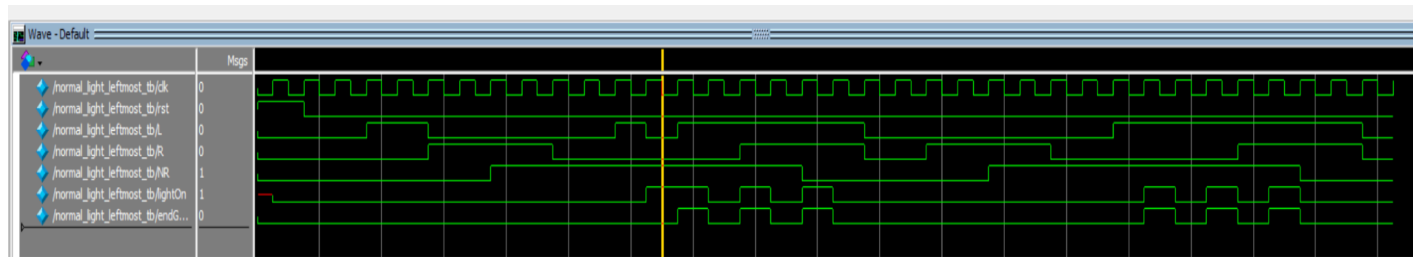
User_input module model sim. As we can see, for the pressed signal at the bottom only activates for a single clock cycle – a pulse. The delay is brought by the flip-flops in the synchronizer

Q3. Lights modules



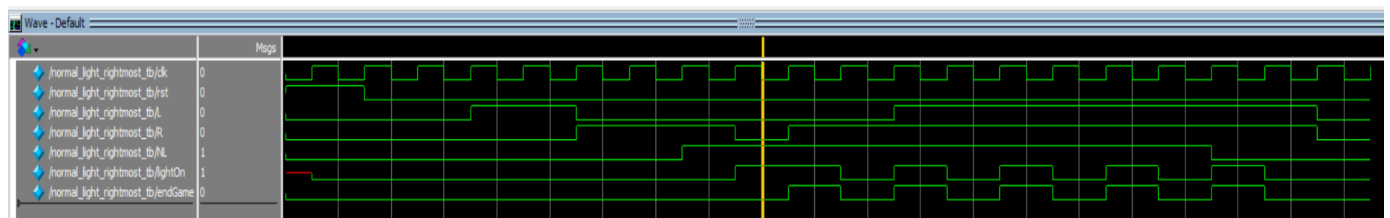
Above is a screenshot of the FSMs and input patterns for each. LED1 and LED9 needed their own (according to me at least) since they only activate in single instances. LED2-8 behave identically except LED5 which only differs by it being active when the circuit is reset which is shown in the bottom right

LED 9 - leftmost light



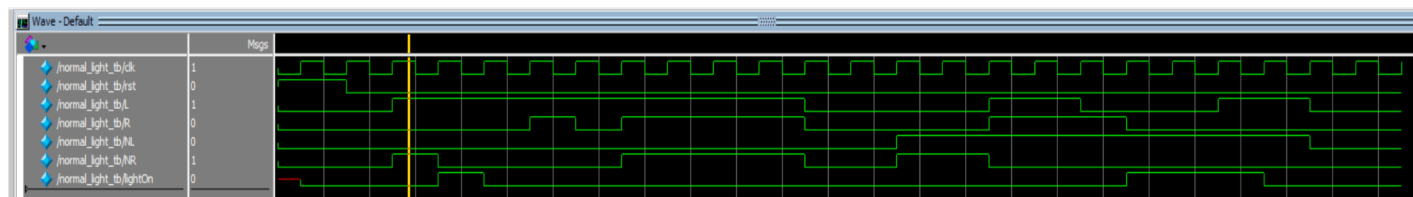
There are a range of combinations of inputs that can be fed into the LED1 module, but there is a very specific condition in which it will turn on which is shown in the FSM diagram. As we can see, the endGame condition only activates when the specific input requirement is met - left input when LED1 is on.

LED 1 - rightmost light

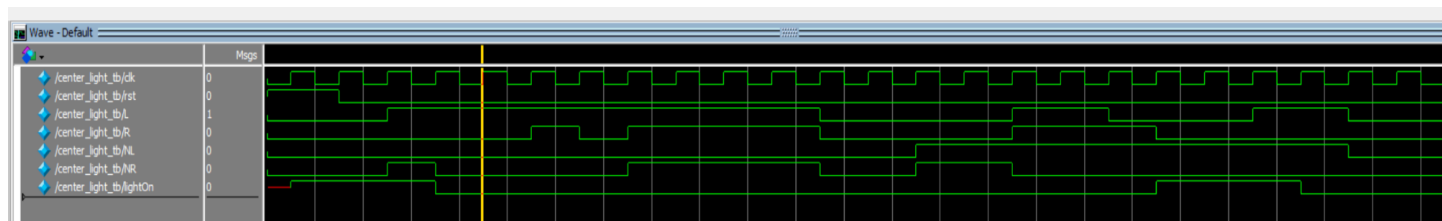


Same thing here, but in the opposite direction to LED 1

LED 2-8 (except 5)

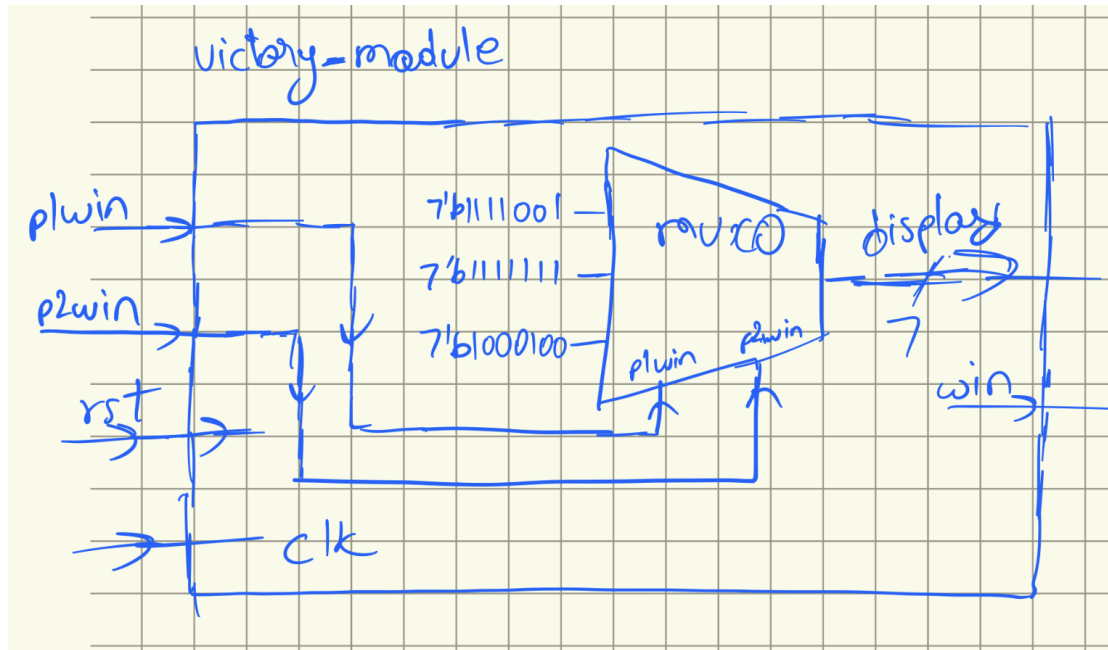


Above is the overall modelsim for all the leds from 2 to 8 except 5. LEDs 2-8 activate only on a specific pair of inputs – basically the LED1 and LED9 input combinations of L, R, NR, and NL. All others do nothing. Here, we can see one of the scenarios where the LED would turn on – when its right neighbor is on and left input is detected (the cursor doesn't show NR being 0, but its effect lasts for a single clock cycle due to delay). At reset, normal LED is also off, which is also shown by modelsim.

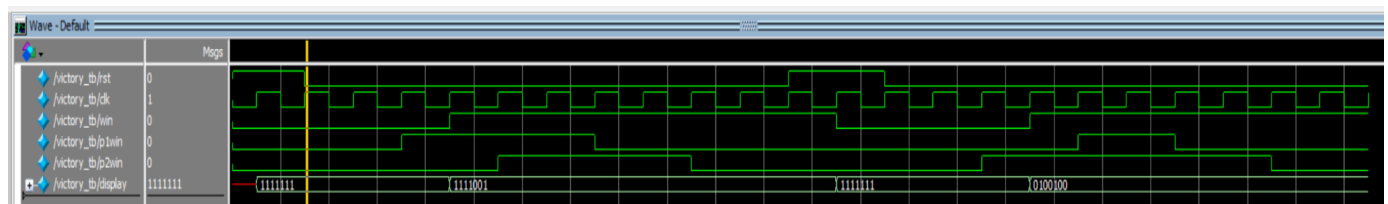


This shows simulated behavior of LED5. It is practically identical to the behavior of the LEDs except for the fact that it is activated when the circuit is reset, which we can see with the lightOn being 1 in the beginning of this simulation.]

Q4. Victory module



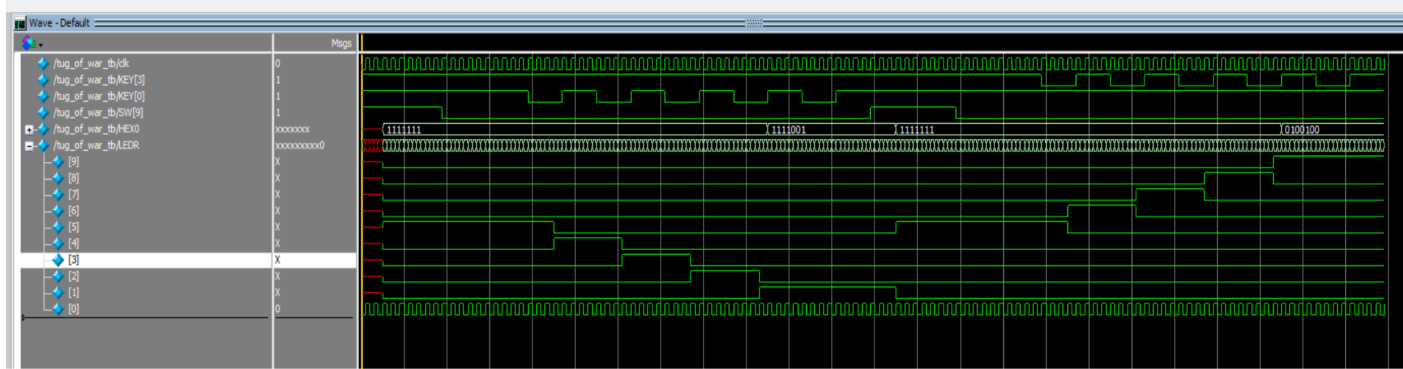
This module checks the state of LEDR1 and LEDR9 every clock cycle to check whether or not someone won. If 1 wins, it outputs a 1 on the hex display through the display signal or 2 if player 2 wins. The win signal is a system wide signal that represents the state of the game. If somebody wins, we don't want them to be able to continue moving the "rope" around since a win is a win - until the game is reset by the SW[9] hardware input aka the reset signal.



Here, we see that at first our hex display is empty (active-low HEX displays) initially. Then, when one of the players wins, the module outputs that player's number to the hex display as well as the state maintaining the win signal which makes sure that erroneous inputs after the game has ended do not affect the playing field or the winner being displayed, only when the game is reset. When the game is reset, everything goes back to default settings and the game can be played once again.

Q5. Tug of war modelsim

Overall:



This is the tug or war module which shows the entire module working in the simulated manner. Initially, the hex display is off, but after enough key strokes, the hex display outputs the value of 3 – seen in the beginning of the simulation. This is also shown for key0 input too resulting in 1 being displayed to the hex display.

Q6. Resource Utilization

Analysis & Synthesis Resource Utilization by Entity										
<<Filter>>										
	Compilation Hierarchy Node	Combinational ALUTs	Dedicated Logic Registers	Block Memory Bits	DSP Blocks	Pins	Virtual Pins	Full Hierarchy Name	Entity Name	Library Name
1	√ tug_of_war	24 (0)	21 (2)	0	0	32	0	[tug_of_war	tug_of_war	work
1	[center_lightlight5]	1 (1)	1 (1)	0	0	0	0	[tug_of_war]center_lightlight5	center_light	work
2	[normal_lightlight2]	1 (1)	1 (1)	0	0	0	0	[tug_of_war]normal_lightlight2	normal_light	work
3	[normal_lightlight3]	1 (1)	1 (1)	0	0	0	0	[tug_of_war]normal_lightlight3	normal_light	work
4	[normal_lightlight4]	1 (1)	1 (1)	0	0	0	0	[tug_of_war]normal_lightlight4	normal_light	work
5	[normal_lightlight6]	1 (1)	1 (1)	0	0	0	0	[tug_of_war]normal_lightlight6	normal_light	work
6	[normal_lightlight7]	1 (1)	1 (1)	0	0	0	0	[tug_of_war]normal_lightlight7	normal_light	work
7	[normal_lightlight8]	1 (1)	1 (1)	0	0	0	0	[tug_of_war]normal_lightlight8	normal_light	work
8	[normal_light_leftmostleftmost]	2 (2)	1 (1)	0	0	0	0	[tug_of_war]normal_light_leftmostleftmost	normal_light_leftmost	work
9	[normal_light_rightmostrightmost]	2 (2)	1 (1)	0	0	0	0	[tug_of_war]normal_light_rightmostrightmost	normal_light_rightmost	work
10	√ [user_inputinput1]	4 (0)	3 (0)	0	0	0	0	[tug_of_war]user_inputinput1	user_input	work
1	[pulsepulse_detector]	2 (2)	1 (1)	0	0	0	0	[tug_of_war]user_inputinput1[pulsepulse_detector]	pulse	work
2	[synchsynchronizer]	2 (2)	2 (2)	0	0	0	0	[tug_of_war]user_inputinput1[synchsynchronizer]	synch	work
11	√ [user_inputinput2]	4 (0)	3 (0)	0	0	0	0	[tug_of_war]user_inputinput2	user_input	work
1	[pulsepulse_detector]	2 (2)	1 (1)	0	0	0	0	[tug_of_war]user_inputinput2[pulsepulse_detector]	pulse	work
2	[synchsynchronizer]	2 (2)	2 (2)	0	0	0	0	[tug_of_war]user_inputinput2[synchsynchronizer]	synch	work
12	[victorywinner_detector]	5 (5)	4 (4)	0	0	0	0	[tug_of_war]victorywinner_detector	victory	work

Time Spent: 18 hours